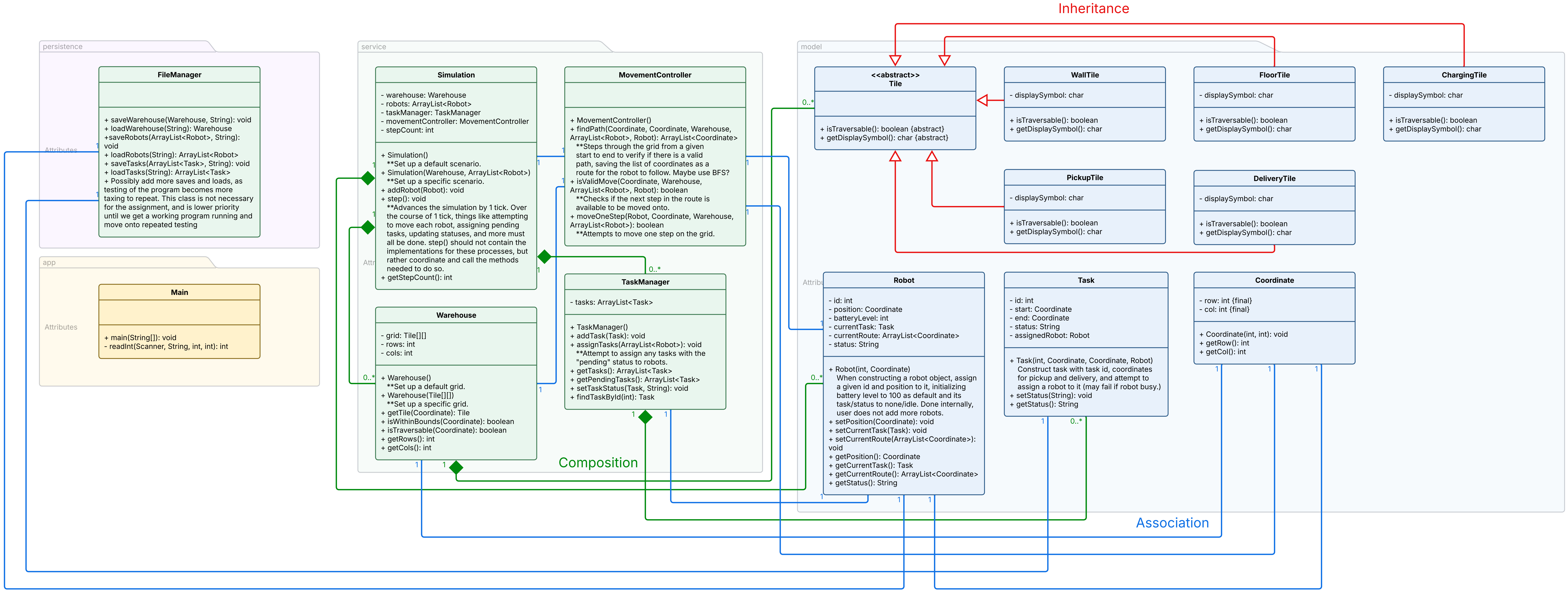

FileManager

```
+ saveWarehouse(Warehouse, String): void
+ loadWarehouse(String): Warehouse
+saveRobots(ArrayList<Robot>, String):
void
+ loadRobots(String): ArrayList<Robot>
+ saveTasks(ArrayList<Task>, String): void
+ loadTasks(String): ArrayList<Task>
+ Possibly add more saves and loads, as
testing of the program becomes more
taxing to repeat. This class is not necessary
for the assignment, and is lower priority
until we get a working program running and
move onto repeated testing
```

Main

```
+ main(String[]): void
- readInt(Scanner, String, int, int): int
```

Simulation

```
- warehouse: Warehouse
- robots: ArrayList<Robot>
- taskManager: TaskManager
- movementController: MovementController
- stepCount: int
```

```
+ Simulation()
  **Set up a default scenario.
+ Simulation(Warehouse, ArrayList<Robot>)
  **Set up a specific scenario.
+ addRobot(Robot): void
+ step(): void
  **Advances the simulation by 1 tick. Over
  the course of 1 tick, things like attempting
  to move each robot, assigning pending
  tasks, updating statuses, and more must
  all be done. step() should not contain the
  implementations for these processes, but
  rather coordinate and call the methods
  needed to do so.
+ getStepCount(): int
```

MovementController

```
+ MovementController()
+ findPath(Coordinate, Coordinate, Warehouse,
ArrayList<Robot>, Robot): ArrayList<Coordinate>
  **Steps through the grid from a given
  start to end to verify if there is a valid
  path, saving the list of coordinates as a
  route for the robot to follow. Maybe use BFS?
+ isValidMove(Coordinate, Warehouse,
ArrayList<Robot>, Robot): boolean
  **Checks if the next step in the route is
  available to be moved onto.
+ moveOneStep(Robot, Coordinate, Warehouse,
ArrayList<Robot>): boolean
  **Attempts to move one step on the grid.
```

Warehouse

```
- grid: Tile[][]
- rows: int
- cols: int
```

```
+ Warehouse()
  **Set up a default grid.
+ Warehouse(Tile[][])
  **Set up a specific grid.
+ getTile(Coordinate): Tile
+ isWithinBounds(Coordinate): boolean
+ isTraversable(Coordinate): boolean
+ getRows(): int
+ getCols(): int
```

TaskManager

```
- tasks: ArrayList<Task>
```

```
+ TaskManager()
+ addTask(Task): void
+ assignTasks(ArrayList<Robot>): void
  **Attempt to assign any tasks with the
  "pending" status to robots.
+ getTasks(): ArrayList<Task>
+ getPendingTasks(): ArrayList<Task>
+ setTaskStatus(Task, String): void
+ findTaskById(int): Task
```

<<abstract>>
Tile

```
+ isTraversable(): boolean {abstract}
+ getDisplaySymbol(): char {abstract}
```

WallTile

```
- displaySymbol: char
```

```
+ isTraversable(): boolean
+ getDisplaySymbol(): char
```

FloorTile

```
- displaySymbol: char
```

```
+ isTraversable(): boolean
+ getDisplaySymbol(): char
```

ChargingTile

```
- displaySymbol: char
```

```
+ isTraversable(): boolean
+ getDisplaySymbol(): char
```

PickupTile

```
- displaySymbol: char
```

```
+ isTraversable(): boolean
+ getDisplaySymbol(): char
```

DeliveryTile

```
- displaySymbol: char
```

```
+ isTraversable(): boolean
+ getDisplaySymbol(): char
```

Robot

```
- id: int
- position: Coordinate
- batteryLevel: int
- currentTask: Task
- currentRoute: ArrayList<Coordinate>
- status: String
```

```
+ Robot(int, Coordinate)
  When constructing a robot object, assign
  a given id and position to it, initializing
  battery level to 100 as default and its
  task/status to none/idle. Done internally,
  user does not add more robots.
+ setPosition(Coordinate): void
+ setCurrentTask(Task): void
+ setCurrentRoute(ArrayList<Coordinate>):
void
+ getPosition(): Coordinate
+ getCurrentTask(): Task
+ getCurrentRoute(): ArrayList<Coordinate>
+ getStatus(): String
```

Task

```
- id: int
- start: Coordinate
- end: Coordinate
- status: String
- assignedRobot: Robot
```

```
+ Task(int, Coordinate, Coordinate, Robot)
  Construct task with task id, coordinates
  for pickup and delivery, and attempt to
  assign a robot to it (may fail if robot busy.)
+ setStatus(String): void
+ getStatus(): String
```

Coordinate

```
- row: int {final}
- col: int {final}
```

```
+ Coordinate(int, int): void
+ getRow(): int
+ getCol(): int
```